

Ligate Chain vs Ethereum Attestation Service

Stefan Stefanović, Ligate Labs

2026-05-25

Ligate Chain vs Ethereum Attestation Service

A Technical Comparison Across Six Architectural Axes

Ligate Labs Research, Working Paper v0.1

Date: 2026-05-25

Status: v0.1 outline. Section structure and intent established; substantive content lands at v0.2.
No formal claims yet.

Contact: hello@ligate.io

Version history: v0.1 (2026-05-25, outline).

Contents

Ligate Chain vs Ethereum Attestation Service	1
A Technical Comparison Across Six Architectural Axes	1
Abstract	3
1. Introduction	3
1.1 Why this comparison exists	3
1.2 Why now	3
1.3 What the note answers	3
1.4 The central question	3
1.5 Approach in brief	3
1.6 Contributions	3
1.7 Scope and non-goals	4
1.8 Document structure	4
2. EAS Architecture in Brief	4
2.1 The two-contract design	4
2.2 Tokenless economic model	4
2.3 Schema collaboration model	5
2.4 SDK and deployment footprint	5
3. Ligate Chain Attestation in Brief	5
3.1 Attestation as a chain operation	5
3.2 PoUA economic security	5
3.3 Threshold attestor sets	5
3.4 Per-schema fee markets	5
3.5 Typed cross-schema composition	6
3.6 Schema-bound tokens	6
3.7 Time-locked attestations	6
4. Six-Axis Comparison	6
4.1 The comparison table	6
4.2 Axis 1: Economic security	7
4.3 Axis 2: Signer model	7
4.4 Axis 3: Fee market	7
4.5 Axis 4: Composition	7
4.6 Axis 5: Token primitives	7
4.7 Axis 6: Time semantics	7
5. Use-Case Profile Mapping	7
5.1 Use cases that lean EAS	7
5.2 Use cases that lean Ligate	8
5.3 Mixed-use scenarios	8
6. Composition Rather Than Competition	8
6.1 Cross-chain typed reference sketch	8
6.2 Why this is interesting	8
6.3 Caveats	8
7. Conclusion	8
References	9
Appendix A: EAS deployment footprint snapshot	9

Abstract

[v0.1: one paragraph stating the comparison axis. EAS is a contract-based attestation registry on Ethereum and major L2s; Ligate Chain is an attestation-native chain with PoUA-backed economic security. Both serve the “signed claim posted on-chain” surface, but they make different architectural commitments. This note compares them across six axes (economic security, signer model, fee market, composition, token primitives, time semantics) and concludes that they sit in different architectural families rather than competing for the same use cases. Composition between the two is sketched but deferred to a cross-chain follow-up.]

1. Introduction

1.1 Why this comparison exists

[v0.1: EAS is the closest direct peer to Ligate Chain in the attestation space. Investors, design partners, and EAS users evaluating Ligate ask the same first-order question: “why not just use EAS?” This note answers it. The position is not that one system replaces the other; the position is that the two systems make different architectural commitments and suit different use-case profiles.]

1.2 Why now

[v0.1: EAS adoption has reached non-trivial scale on Ethereum L1 and Base / Optimism / Arbitrum. EU AI Act Article 50 (full effect August 2026) and the C2PA adoption wave have moved attestation infrastructure from niche to category. The question of which substrate to attest on is suddenly mainstream. This note belongs to that moment.]

1.3 What the note answers

[v0.1: Three questions: (a) where do EAS and Ligate make the same architectural commitment, (b) where do they diverge, and (c) what use cases each is better suited to. The answer is comparison, not advocacy.]

1.4 The central question

[v0.1: Is an attestation primitive better deployed as a smart contract on a general-purpose chain, or as a native operation on a chain whose entire surface is attestations? This note frames the trade-off across six axes and concludes that the answer depends on the use case’s economic-security needs and composition demands.]

1.5 Approach in brief

[v0.1: Two surveys (EAS, Ligate) followed by axis-by-axis comparison. No formal proof; the note is descriptive, not derivative. Composition sketch (§7) is conditional on the cross-chain follow-up paper landing.]

1.6 Contributions

1. **EAS architecture survey.** Brief technical description of the two-contract design, schema registry, attestation lifecycle, fee structure, SDK surface.

2. **Six-axis comparison.** Economic security, signer model, fee market, composition, token primitives, time semantics, presented in a single table plus per-axis discussion.
3. **Use-case profile mapping.** Which workflows lean toward EAS, which lean toward Ligate, with honest qualifications.
4. **Composition sketch.** How a Themisra attestation could reference an EAS attestation via CSC v0.2 §4.3 typed reference + cross-chain bridge (deferred to follow-up paper).

1.7 Scope and non-goals

In scope:

- Technical architecture of EAS as deployed in 2026
- Architectural commitments of Ligate Chain v0.2+ relative to EAS
- Use-case profile mapping (qualitative)
- Composition possibilities (conditional on follow-up paper)

Explicitly out of scope:

- Quantitative benchmarking (per-attestation cost, throughput) — would require empirical measurement; v0.3+ if useful
- Detailed cross-chain bridge specification — lives in #136
- Argument that EAS is broken or insufficient for its use cases — it is not
- Migration guide from EAS to Ligate — the note is comparison, not advocacy

1.8 Document structure

[v0.1: §2 surveys EAS architecture. §3 surveys Ligate Chain’s attestation primitive. §4 presents the six-axis comparison table and per-axis discussion. §5 maps use-case profiles. §6 sketches composition possibilities. §7 concludes.]

2. EAS Architecture in Brief

[v0.1: Two-contract design: schema registry (schema declarations) + attestation contract (attestation submissions referencing schemas). Tokenless: no native token, fees are L1 / L2 gas only. Permissionless: any address can register a schema, any address can attest. Schema collaboration via community proposals off-chain. SDKs in TypeScript and Rust. Deployed across Ethereum L1 and major L2s (Base, Optimism, Arbitrum, others). Revocation is supported at the attestation level. Off-chain attestation variant exists with signed EIP-712 messages anchored on-chain by reference.]

2.1 The two-contract design

[v0.1: Brief description of schema registry contract + attestation contract. UID-based addressing. Schema field encoding via solidity-style type strings. Reference attestations supported via UID pointers (not typed).]

2.2 Tokenless economic model

[v0.1: No EAS-native token. Costs are L1 / L2 gas at the time of submission. Schema authors do not capture fee share. Validators (Ethereum / L2 validators) capture gas as normal. Implications:

EAS is “infrastructure public good” framed, but the implications cascade into the signer model (anyone can attest for the cost of gas) and the economic security model (no slashing of attestors).]

2.3 Schema collaboration model

[v0.1: Community schema proposals via off-chain coordination (forum, GitHub, attestor-org governance). No protocol-level canonical schema mechanism. Implications: schema namespacing is by convention, not enforced.]

2.4 SDK and deployment footprint

[v0.1: SDK in TypeScript + Rust. Deployed on Ethereum L1 + Base + Optimism + Arbitrum + others (cite docs at authoring time). Each deployment is a separate contract, separate registry, no cross-deployment composition at the contract level.]

3. Ligate Chain Attestation in Brief

[v0.1: Attestation as a first-class chain operation (`RegisterSchema`, `RegisterAttestorSet`, `SubmitAttestation`). Native \$AVOW token captures fees. PoUA consensus weighting (Lemma 1 cost-to-grind floor) provides economic security on attestor behavior. Threshold attestor sets as native primitive. Per-schema fee markets (EIP-1559-style per-schema). Typed cross-schema composition with cascade semantics. Schema-bound tokens with threshold mint authority. Time-locked attestations (commit-reveal as runtime primitive). Quoted from parent papers; not re-derived here.]

3.1 Attestation as a chain operation

[v0.1: Three native ops: `RegisterSchema` (anyone can register), `RegisterAttestorSet` (anyone can register with threshold parameters), `SubmitAttestation` (signed by attestor set’s quorum). Quote from the PoUA paper §2.]

3.2 PoUA economic security

[v0.1: Lemma 1 cost-to-grind floor: $F^{\text{net}} \geq \tau_{\text{burn}} \cdot \Delta r / (\eta \cdot \alpha)$. The bound is signature-scheme-agnostic and primitive-agnostic. Quote from PoUA v0.9.2 §3.]

3.3 Threshold attestor sets

[v0.1: Attestor sets are first-class chain entities with declared threshold t of n members. `SubmitAttestation` requires t valid signatures from set members. Set membership updates are themselves attestations. Quote from PoUA §2.4.]

3.4 Per-schema fee markets

[v0.1: Per-schema base fee b_σ with EIP-1559 dynamics on per-schema utilization. PoUA-coupled burn floor preserved per-schema. Quote from per-schema-fees v0.2 §4.]

3.5 Typed cross-schema composition

[**v0.1:** Schemas declare input-type sets; references are typed and runtime-checked at admission. Cascade semantics on input invalidation. Quote from CSC v0.2 §3 + §4.]

3.6 Schema-bound tokens

[**v0.1:** Token mints are attestations under canonical schema `chain.token-mint/v1` with threshold mint authority bound to an attester set. Quote from SBT v0.2 §2 + §3.]

3.7 Time-locked attestations

[**v0.1:** Commit-reveal as runtime primitive: `MsgCommit` + `MsgReveal` + state machine. Quote from time-locked-attestations v0.2 §3 + §4.]

4. Six-Axis Comparison

[**v0.1:** This section presents the comparison table + per-axis discussion. The table is the load-bearing artifact of the note. At v0.2 every cell carries a citation or a documented EAS behavior link.]

4.1 The comparison table

[**v0.1:** Table to be drafted. Six rows (one per axis), three columns (EAS, Ligate, Notes).]

Axis	EAS	Ligate Chain	Notes
Economic security	[v0.1: none beyond L1/L2 gas]	[v0.1: PoUA Lemma 1 floor]	[v0.1: this is the central differentiator]
Signer model	[v0.1: single-sig per attestation]	[v0.1: threshold attester set]	[v0.1: EAS bolts on multisig; Ligate is native]
Fee market	[v0.1: single L1/L2 gas market]	[v0.1: per-schema EIP-1559]	[v0.1: Ligate prices high-value schemas higher]
Composition	[v0.1: untyped UID pointers]	[v0.1: typed refs with cascade]	[v0.1: Ligate enforces type-correctness at admission]
Token primitives	[v0.1: none]	[v0.1: SBT with threshold mint]	[v0.1: EAS doesn't try; Ligate ships SBT]
Time semantics	[v0.1: none beyond block time]	[v0.1: TLA commit-reveal primitive]	[v0.1: Ligate has commit-reveal as runtime; EAS at app layer]

4.2 Axis 1: Economic security

[v0.1: Per-axis discussion. EAS has no economic security at the attestation primitive level; anyone can attest for the cost of L1/L2 gas. Misbehavior consequence is reputational at the application layer. Ligate has PoUA Lemma 1 cost-to-grind floor as the economic-security primitive at the consensus layer. Misbehavior carries reputation-loss + fee-burn consequences at the chain layer.]

4.3 Axis 2: Signer model

[v0.1: Per-axis discussion. EAS attestations are signed by one address. Threshold-attestor patterns are bolt-on multisig contracts. Ligate attestor sets are first-class chain entities with declared threshold t of n .]

4.4 Axis 3: Fee market

[v0.1: Per-axis discussion. EAS shares one gas market across all attestations. Ligate per-schema EIP-1559 lets high-value schemas (e.g., regulated currency, AI-provenance) price out spam without affecting other schemas.]

4.5 Axis 4: Composition

[v0.1: Per-axis discussion. EAS attestations can reference each other by UID but the reference is untyped; consumer code must validate types off-chain. Ligate's typed cross-schema composition (CSC v0.2 §4.3) enforces type correctness at admission time and cascades invalidation through the dependency graph.]

4.6 Axis 5: Token primitives

[v0.1: Per-axis discussion. EAS does not ship a token primitive (intentionally; EAS is attestation-only). Ligate ships schema-bound tokens (SBT v0.2) with threshold mint authority. Different scope claims.]

4.7 Axis 6: Time semantics

[v0.1: Per-axis discussion. EAS has no commit-reveal or time-lock primitive; applications layer this at the contract layer if needed. Ligate ships time-locked attestations (TLA v0.2) as a runtime primitive.]

5. Use-Case Profile Mapping

[v0.1: This section maps representative use cases to the system whose architectural commitments fit them. The framing is honest: some use cases lean EAS for simplicity reasons, some lean Ligate for economic-security reasons.]

5.1 Use cases that lean EAS

[v0.1: Workflows where simplicity dominates: single-signer attestations, no threshold mint authority, no per-schema fee-market needs, no cross-schema composition demands. Examples: simple credentialing, single-issuer badge attestations, low-stakes onchain reputation.]

5.2 Use cases that lean Ligate

[v0.1: Workflows where economic security or threshold authority dominates: regulated currency issuance, high-value AI-provenance with permanent receipts, audit-bearing attestations where misbehavior must carry economic consequence, cross-schema workflows that need typed composition. Examples: schema-bound currency, AI-content provenance under EU AI Act scrutiny, DAO treasury attestations, regulated-license registrations.]

5.3 Mixed-use scenarios

[v0.1: Workflows where a single product surface might span both architectures. Example: a Themisra AI-provenance attestation on Ligate referencing an EAS-attested model-card on Ethereum. The composition is sketched in §6.]

6. Composition Rather Than Competition

[v0.1: The note's framing is that EAS and Ligate sit in different architectural families and can compose. A Themisra attestation could carry a typed reference (CSC v0.2 §4.3) to an EAS attestation via a cross-chain bridge. The bridge mechanics are out of scope for this note and live in the cross-chain attestation portability follow-up paper.]

6.1 Cross-chain typed reference sketch

[v0.1: Brief sketch. A Themisra attestation declares an input type `eas.attestation.<schema-uid>` and an input value (`chain-id`, `eas-uid`, `light-client-proof`). The admission check verifies the light-client proof against a known Ethereum / L2 header. Cascade semantics apply if the EAS attestation is revoked, subject to revocation-visibility latency across the IBC round-trip.]

6.2 Why this is interesting

[v0.1: Existing EAS deployments could be referenced by Ligate-side attestations without re-issuing them on Ligate. This preserves EAS's existing data while letting Ligate-side workflows add typed composition, threshold attestor mint authority, recall semantics, etc. on top.]

6.3 Caveats

[v0.1: Bridge complexity, IBC update latency, revocation-visibility latency, security inheritance from the weaker chain. Each documented in the cross-chain follow-up paper.]

7. Conclusion

[v0.1: Two paragraphs. (1) The comparison is across architectural families, not feature lists. EAS makes infrastructure-public-good commitments; Ligate makes economic-security-floor commitments. Both are defensible positions. (2) Composition between the two is plausible and interesting. The cross-chain follow-up paper is the next step. Until then, this note positions Ligate accurately without overclaiming.]

References

[v0.1: References to fill in at v0.2. Anchors:]

1. EAS (Ethereum Attestation Service) official documentation. <https://docs.attest.org/>
2. EAS specification repository. <https://github.com/ethereum-attestation-service>
3. Sign Protocol (EAS-variant attestation network).
4. Verax (Ethereum attestation registry, Linea-flavored).
5. PoUA paper (this repo, [papers/poua/](#)).
6. Per-Schema Fees paper (this repo, [papers/per-schema-fees/](#)).
7. Cross-Schema Composition paper (this repo, [papers/cross-schema-composition/](#)).
8. Schema-Bound Tokens paper (this repo, [papers/schema-bound-tokens/](#)).
9. Time-Locked Attestations paper (this repo, [papers/time-locked-attestations/](#)).

Appendix A: EAS deployment footprint snapshot

[v0.1: At v0.2: a tabular snapshot of EAS deployments as of authoring date. Per-chain attestation counts (if publicly accessible). Schema counts. Adoption signals.]